Call the _layout using: (@{

```
    Layout = "~/Views/Shared/_Layout.cshtml";
})
```

)(@using Microsoft.AspNetCore.Http
@inject IHttpContextAccessor HttpContextAccessor

@{
    var role = HttpContextAccessor.HttpContext?.Session.GetString("UserRole");
}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - AxertLMS</title>
    <link rel="stylesheet" href="~/css/site.css" />
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">

    <style>
        /* Modern Dark Mode Design System Constants */
        :root {
            --bg-main: #161719;
            --bg-sidebar: #0F1012;
            --text-primary: #FFFFFF;
            --text-secondary: #8E94A0;
            --text-muted: #5C616C;
            --accent-glow: rgba(255, 255, 255, 0.03);
            --border-subtle: rgba(255, 255, 255, 0.06);
            --sidebar-width: 235px;
        }

        body {
            margin: 0;
            font-family: 'Segoe UI', system-ui, -apple-system, sans-serif;
            background-color: var(--bg-main);
            color: var(--text-primary);
            -webkit-font-smoothing: antialiased;
            -moz-osx-font-smoothing: grayscale;
        }

        .app-container {
            display: flex;
            min-height: 100vh;

```
}

/* PREMIUM SIDEBAR ARCHITECTURE */
.sidebar {
    width: var(--sidebar-width);
    background: var(--bg-sidebar);
    display: flex;
    flex-direction: column;
    border-right: 1px solid var(--border-subtle);
    position: fixed;
    top: 0;
    left: 0;
    height: 100vh;
    z-index: 100;
    box-sizing: border-box;
}

/* LOGO BOX - Proportional spacing adjustments */
.sidebar-logo-container {
    padding-top: 40px;
    padding-right: 60px;
    padding-bottom: 20px;
    padding-left: 50px;
    display: flex;
    justify-content: flex-start;
    align-items: center;
    border-bottom: 1px solid var(--border-subtle);
}

.sidebar-logo {
    width: 133px;
    height: auto;
    object-fit: contain;
    filter: drop-shadow(0px 4px 8px rgba(0, 0, 0, 0.3));
}

/* NAVIGATION INTERFACE */
.nav-scroller {
    flex: 1;
    overflow-y: auto;
    padding: 20px 0;
}

    /* Scrollbar styling for a completely premium look */
```

```
    .nav-scroller::-webkit-scrollbar {
        width: 5px;
    }

    .nav-scroller::-webkit-scrollbar-thumb {
        background: rgba(255, 255, 255, 0.1);
        border-radius: 4px;
    }

.nav-group {
    padding: 24px 28px 8px;
    font-size: 0.72rem;
    text-transform: uppercase;
    color: var(--text-muted);
    letter-spacing: 1.5px;
    font-weight: 700;
}

.nav-link {
    display: flex;
    align-items: center;
    padding: 13px 28px;
    color: var(--text-secondary);
    text-decoration: none;
    font-size: 0.92rem;
    font-weight: 500;
    border-left: 3px solid transparent;
    transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
}

    .nav-link:hover {
        background: var(--accent-glow);
        color: var(--text-primary);
        border-left-color: var(--text-secondary);
        padding-left: 32px; /* Smooth elegant shift forward */
    }

.nav-icon {
    margin-right: 14px;
    font-size: 1.05rem;
    width: 20px; /* Fixed footprint keeps column layout completely aligned */
    text-align: center;
    opacity: 0.8;
    transition: transform 0.2s ease;
```

```
        }

        .nav-link:hover .nav-icon {
            opacity: 1;
            transform: scale(1.05);
        }

        /* ACTIONS FOOTER */
        .sidebar-footer {
            padding: 15px 0 25px;
            border-top: 1px solid var(--border-subtle);
        }

        .nav-link.logout {
            color: #F87171;
        }

            .nav-link.logout:hover {
                border-left-color: #EF4444;
                color: #EF4444;
                background: rgba(239, 68, 68, 0.03);
            }

        /* APP CANVAS VIEWPORT */
        .main-content {
            flex: 1;
            padding: 40px;
            margin-left: var(--sidebar-width);
            background-color: var(--bg-main);
            box-sizing: border-box;
            min-height: 100vh;
        }
    </style>
</head>
<body>
    <div class="app-container">
        @if (role != null)
        {
            <div class="sidebar">
                <div class="sidebar-logo-container">
                    <img src="~/images/logo.png" alt="LMS Logo" class="sidebar-logo" />
                </div>

                <div class="nav-scroller">
```

```
                @if (role == "Student")
                {
                    <div class="nav-group">Academic Portal</div>
                    <a href="/Student/Dashboard" class="nav-link"><i class="fa-solid fa-chart-pie
nav-icon"></i> Dashboard</a>
                    <a href="/Student/Classrooms" class="nav-link"><i class="fa-solid
fa-chalkboard-user nav-icon"></i> My Classrooms</a>
                    <a href="/Student/Calendar" class="nav-link"><i class="fa-solid fa-calendar-days
nav-icon"></i> Calendar</a>
                    <a href="/Student/Assignments" class="nav-link"><i class="fa-solid
fa-file-signature nav-icon"></i> Assignments</a>
                    <a href="/Student/Announcements" class="nav-link"><i class="fa-solid
fa-bullhorn nav-icon"></i> Announcements</a>
                }
                else if (role == "Teacher")
                {
                    <div class="nav-group">Faculty Workspace</div>
                    <a href="/Teacher/Dashboard" class="nav-link"><i class="fa-solid fa-chart-pie
nav-icon"></i> Dashboard</a>
                    <a href="/Teacher/MyClasses" class="nav-link"><i class="fa-solid fa-chalkboard
nav-icon"></i> My Classes</a>
                    <a href="/Teacher/GradingCenter" class="nav-link"><i class="fa-solid
fa-graduation-cap nav-icon"></i> Grading Center</a>
                }
                else if (role == "Registrar")
                {
                    <div class="nav-group">Operations Overview</div>
                    <a href="/Registrar/Dashboard" class="nav-link"><i class="fa-solid fa-chart-pie
nav-icon"></i> Dashboard</a>

                    <div class="nav-group">Administration</div>
                    <a href="/Registrar/RegisterStudent" class="nav-link"><i class="fa-solid
fa-user-plus nav-icon"></i> Enroll Student</a>
                    <a href="/Registrar/RegisterTeacher" class="nav-link"><i class="fa-solid
fa-user-tie nav-icon"></i> Add Faculty</a>
                    <a href="/Registrar/ManageSections" class="nav-link"><i class="fa-solid
fa-layer-group nav-icon"></i> Manage Sections</a>
                    <a href="/Registrar/ManageSubjects" class="nav-link"><i class="fa-solid fa-book
nav-icon"></i> Manage Subjects</a>
                }
                else if (role == "Admin")
                {
                    <div class="nav-group">Management Core</div>
```

```
                <a href="/Admin/Dashboard" class="nav-link"><i class="fa-solid fa-gauge-high
nav-icon"></i> Overview</a>
                <a href="/Admin/ManageUsers" class="nav-link"><i class="fa-solid fa-users-gear
nav-icon"></i> Manage Users</a>
            }
            else if (role == "Super Admin")
            {
                <div class="nav-group">System Architecture</div>
                <a href="/SuperAdmin/Dashboard" class="nav-link"><i class="fa-solid
fa-heart-pulse nav-icon"></i> System Health</a>
                <a href="/SuperAdmin/Database" class="nav-link"><i class="fa-solid fa-database
nav-icon"></i> Raw Database</a>
                <a href="/SuperAdmin/RoleManager" class="nav-link"><i class="fa-solid
fa-shield-halved nav-icon"></i> Access Control</a>
            }
        </div>

        <div class="sidebar-footer">
            <a href="/Account/Logout" class="nav-link logout"><i class="fa-solid
fa-right-from-bracket nav-icon"></i> Logout</a>
        </div>
    </div>
}

<div class="main-content">
    @RenderBody()
</div>
</div>
</body>
</html>
)(using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    public class Announcement
    {
        [Key]
        public int announcement_id { get; set; }

        public int classroom_id { get; set; }

        public int user_id { get; set; }
```

```
        public string message { get; set; } = string.Empty;

        public DateTime created_at { get; set; }

        // Navigation properties so EF Core can link the data together
        [ForeignKey("classroom_id")]
        public virtual Classroom? Classroom { get; set; }

        [ForeignKey("user_id")]
        public virtual User? User { get; set; }
    }
}
)
(using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    public class Assignment
    {
        [Key]
        public int assignment_id { get; set; }

        public int classroom_id { get; set; }

        public string title { get; set; } = string.Empty;

        public DateTime due_date { get; set; }

        public string? description { get; set; }

        public string? attachment_link { get; set; }

        // --- THESE ARE THE FIELDS YOUR CONTROLLER IS LOOKING FOR ---
        public string Category { get; set; } = "WW";
        public int MaxScore { get; set; } = 100;
        // ----------------------------------------------------------

        [ForeignKey("classroom_id")]
        public virtual Classroom? Classroom { get; set; }
    }
}
```

```
)
(using System.Collections.Generic;

namespace AxertLMS.Models // Make sure this matches your project's model namespace
{
    public class ClassGradebookViewModel
    {
        public Classroom Classroom { get; set; } = null!;
        public List<Student> Students { get; set; } = new();
        public List<Assignment> Assignments { get; set; } = new();
        public List<Submission> Submissions { get; set; } = new();
        public List<Grade> Grades { get; set; } = new();
    }
}
)
(using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    public class Classroom
    {
        [Key]
        public int classroom_id { get; set; }

        public int subject_id { get; set; }
        [ForeignKey("subject_id")]
        public Subject? Subject { get; set; }

        public int teacher_id { get; set; }
        [ForeignKey("teacher_id")]
        public Teacher? Teacher { get; set; }

        public int section_id { get; set; }
        [ForeignKey("section_id")]
        public Section? Section { get; set; }

        // 👇 THE FIX: Added these navigation lists so the code can fetch them! 👇
        public ICollection<Assignment>? Assignments { get; set; }

        public ICollection<Module>? Modules { get; set; }

        public ICollection<Quiz>? Quizzes { get; set; }
```

```
    }
}
)
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    [Table("Enrollments")]
    public class Enrollment
    {
        [Key]
        public int enrollment_id { get; set; }
        public int student_id { get; set; }
        public int classroom_id { get; set; }
        public DateTime enrollment_date { get; set; }

        // Tells EF Core to use student_id to find the Student
        [ForeignKey("student_id")]
        public Student? Student { get; set; }

        // Tells EF Core to use classroom_id to find the Classroom
        [ForeignKey("classroom_id")]
        public Classroom? Classroom { get; set; }
    }
}
)
(namespace AxertLMS.Models
{
    public class ErrorViewModel
    {
        public string? RequestId { get; set; }

        public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
    }
}

)
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    public class Grade
```

```
    {
        [Key]
        public int grade_id { get; set; }

        [ForeignKey("Submission")]
        public int submission_id { get; set; }
        public virtual Submission? Submission { get; set; }

        [Column(TypeName = "decimal(5,2)")]
        public decimal score { get; set; }

        public string? feedback { get; set; }
    }
}
)
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    public class Module
    {
        [Key]
        public int module_id { get; set; }
        public int classroom_id { get; set; }
        public string title { get; set; } = string.Empty;
        public string description { get; set; } = string.Empty;
        public string file_path { get; set; } = string.Empty;

        [NotMapped] // 👈 THIS IS THE MAGIC TAG. It tells Entity Framework to ignore this for the
database!
        public DateTime created_at { get; set; }

        [ForeignKey("classroom_id")]
        public Classroom? Classroom { get; set; }
    }
}
)
(namespace AxertLMS.Models
{
    public class ProfileViewModel
    {
        // Universal Data
        public int UserId { get; set; }
```

```
        public string Role { get; set; } = string.Empty;
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string Status { get; set; } = string.Empty;

        // Student Specific Data (Nullable, as non-students won't have this)
        public string? StudentNumber { get; set; }
        public string? SectionName { get; set; }
        public string? Strand { get; set; }
        public string? YearLevel { get; set; }
        public int? EnrolledClassesCount { get; set; }

        // Teacher Specific Data (Nullable)
        public string? Specialization { get; set; }
        public int? HandledClassesCount { get; set; }
    }
}
)
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    [Table("Quizzes")] // 👈 FORCES the database table name to be "Quizzes"
    public class Quiz
    {
        [Key]
        public int quiz_id { get; set; }

        public int classroom_id { get; set; }

        public int? module_id { get; set; }

        public string title { get; set; } = string.Empty;

        public string difficulty { get; set; } = "Easy";

        public DateTime created_at { get; set; }

        [ForeignKey("classroom_id")]
        public Classroom? Classroom { get; set; }

        [ForeignKey("module_id")]
```

```
        public Module? Module { get; set; }

        public ICollection<QuizQuestion>? Questions { get; set; }
    }
}
)
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    [Table("QuizQuestions")] // 👈 FORCES the database table name to be "QuizQuestions"
    public class QuizQuestion
    {
        [Key]
        public int question_id { get; set; }

        public int quiz_id { get; set; }

        public string question_type { get; set; } = string.Empty;

        public string question_text { get; set; } = string.Empty;

        public string? option_a { get; set; }

        public string? option_b { get; set; }

        public string? option_c { get; set; }

        public string? option_d { get; set; }

        public string correct_answer { get; set; } = string.Empty;

        [ForeignKey("quiz_id")]
        public Quiz? Quiz { get; set; }
    }
})
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace AxertLMS.Models
{
    [Table("Sections")]
    public class Section
    {
```

```
        [Key]
        public int section_id { get; set; }

        public string section_name { get; set; }

        public string strand { get; set; }

        public string year_lvl { get; set; }
    }
}
)
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    [Table("Students")]
    public class Student
    {
        [Key]
        public int student_id { get; set; }

        public int user_id { get; set; }
        public int section_id { get; set; }
        public string student_number { get; set; }

        // This tells EF Core to use user_id to find the User
        [ForeignKey("user_id")]
        public User? User { get; set; }

        // This tells EF Core to use section_id to find the Section
        [ForeignKey("section_id")]
        public Section? Section { get; set; }
    }

}
)
(using System.Collections.Generic;

namespace AxertLMS.Models
{
    public class StudentViewModel
    {
        // Holds the student's personal info, section, and user details
```

```
        public Student StudentInfo { get; set; }

        // Holds the list of classrooms they are enrolled in
        public List<Classroom> Classrooms { get; set; }
    }
}
)
(namespace AxertLMS.Models
{
    public class Subject
    {
        public int subject_id { get; set; }
        public string subject_name { get; set; }
        public string description { get; set; }
    }
}
)
(using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    public class Submission
    {
        [Key]
        public int submission_id { get; set; }

        public int assignment_id { get; set; }
        public int student_id { get; set; }

        // Existing: For students uploading documents
        public string file_path { get; set; } = string.Empty;

        // NEW: The actual text content for the Plagiarism Engine to scan
        public string? submission_text { get; set; }

        public DateTime submitted_at { get; set; }

        // NEW: Plagiarism Tracking Infrastructure
        public decimal PlagiarismPercentage { get; set; }
        public string? PlagiarismReport { get; set; }

        // Navigation properties
```

```
        [ForeignKey("assignment_id")]
        public Assignment? Assignment { get; set; }

        [ForeignKey("student_id")]
        public Student? Student { get; set; }
    }
}
)
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    [Table("Teachers")]
    public class Teacher
    {
        [Key]
        public int teacher_id { get; set; }
        public int user_id { get; set; }
        public string specialization { get; set; } = string.Empty;

        // Tells EF Core to use user_id to find the User
        [ForeignKey("user_id")]
        public User? User { get; set; }
    }

}

)
(using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AxertLMS.Models
{
    [Table("Users")]
    public class User
    {
        [Key]
        public int user_id { get; set; }
        public string role { get; set; } = string.Empty;
        public string f_name { get; set; } = string.Empty;
        public string l_name { get; set; } = string.Empty;
        public string email { get; set; } = string.Empty;
        public string password { get; set; } = string.Empty;
```

```
        public string status { get; set; } = string.Empty;
    }
}
)
(using AxertLMS.Data;
using AxertLMS.Models;
using AxertLMS.Helpers;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
using System;
using System.Linq;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace AxertLMS.Controllers
{
    public class AccountController : Controller
    {
        private readonly ApplicationDbContext _context;
        private readonly ILogger<AccountController> _logger; // Enforces Section 6 Audit
requirements

        public AccountController(ApplicationDbContext context, ILogger<AccountController>
logger)
        {
            _context = context;
            _logger = logger;
        }

        [HttpGet]
        public IActionResult Register() => View();

        // SATISFIES SECTION 3 & 5: Secure Registration Flow with Strict Input Validation
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Register(User model, string rawPassword)
        {
            if (model == null || string.IsNullOrEmpty(rawPassword))
            {
                ViewBag.Error = "Registration inputs cannot be empty.";
                return View();
            }
```

```
        // Input Validation: Strong Email Check to eliminate injection vectors
        string emailPattern = @"^[^@\s]+@[^@\s]+\.[^@\s]+$";
        if (!Regex.IsMatch(model.email, emailPattern))
        {
            _logger.LogWarning($"VALIDATION_FAILURE: Malformed email registration attempt:
'{model.email}'");
            ViewBag.Error = "Invalid email formatting provided.";
            return View();
        }

        // Access Control: Prevent account duplication
        var existingUser = _context.Users.FirstOrDefault(u => u.email == model.email);
        if (existingUser != null)
        {
            ViewBag.Error = "An account with this email already exists.";
            return View();
        }

        try
        {
            // Encrypt data before storage using high-entropy BCrypt
            model.password = PasswordHelper.Hash(rawPassword);
            model.status = "Active";

            _context.Users.Add(model);
            await _context.SaveChangesAsync();

            _logger.LogInformation($"AUDIT_SUCCESS: New user registered successfully. UID:
{model.user_id}, Role: {model.role}");
            return RedirectToAction("Login");
        }
        catch (Exception ex)
        {
            _logger.LogCritical($"DATABASE_ERROR: Registration persistence failed. Message:
{ex.Message}");
            ViewBag.Error = "An internal error occurred. Please try again later.";
            return View();
        }
    }

    [HttpGet]
    public IActionResult Login() => View();

    // SATISFIES SECTION 3, 5, & 6: Production-Grade Hardened Login Engine
```

```
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Login(string Email, string Password)
        {
            await Task.Delay(1000); // Defense-in-depth: Throttle brute force discovery speeds

            if (string.IsNullOrEmpty(Email) || string.IsNullOrEmpty(Password))
            {
                ViewBag.Error = "Please complete all mandatory login fields.";
                return View();
            }

            // Strict Validation parsing
            if (!Regex.IsMatch(Email, @"^[^@\s]+@[^@\s]+\.[^@\s]+$"))
            {
                _logger.LogWarning($"SECURITY_ALERT: Malformed login username structure
attempted: '{Email}'");
                ViewBag.Error = "Invalid email or password. Please try again."; // Generic error masks
internal database presence
                return View();
            }

            var user = _context.Users.FirstOrDefault(u => u.email == Email);
            if (user == null)
            {
                _logger.LogWarning($"AUTH_FAILURE: Account target not found for email: {Email}");
                ViewBag.Error = "Invalid email or password. Please try again.";
                return View();
            }

            bool isPasswordValid = false;
            bool needsBcryptUpgrade = false;

            // Opportunistic Hashing Engine (Automated Legacy Account Modernization)
            if (user.password != null && user.password.Length == 64 &&
!user.password.StartsWith("$"))
            {
                string hashedInput = PasswordHelper.HashSHA256(Password);
                if (hashedInput == user.password)
                {
                    isPasswordValid = true;
                    needsBcryptUpgrade = true;
                }
            }
```

```
        else
        {
            isPasswordValid = PasswordHelper.VerifyBCrypt(Password, user.password, _logger);
        }

        if (!isPasswordValid)
        {
            _logger.LogWarning($"AUTH_FAILURE: Incorrect password password submission for
identity: {Email}");
            ViewBag.Error = "Invalid email or password. Please try again.";
            return View();
        }

        if (needsBcryptUpgrade)
        {
            user.password = PasswordHelper.Hash(Password);
            await _context.SaveChangesAsync();
            _logger.LogInformation($"MIGRATION_AUDIT: System user '{Email}' updated
transparently from SHA-256 to BCrypt.");
        }

        if (user.status == "Inactive")
        {
            _logger.LogWarning($"AUTH_RESTRICTION: Deactivated identity login blocked for
profile: {Email}");
            ViewBag.Error = "Your account is currently inactive. Please contact the Registrar.";
            return View();
        }

        // Hardened Session Lifecycle Management (Protects against Session Fixation)
        HttpContext.Session.Clear();
        HttpContext.Session.SetInt32("UserId", user.user_id);
        HttpContext.Session.SetString("UserRole", user.role);
        HttpContext.Session.SetString("UserEmail", user.email);

        _logger.LogInformation($"AUTH_SUCCESS: Secure session authorization granted for
identity: {Email} (Role: {user.role})");

        switch (user.role)
        {
            case "Teacher": return RedirectToAction("Dashboard", "Teacher");
            case "Student": return RedirectToAction("Dashboard", "Student");
            case "Registrar": return RedirectToAction("Dashboard", "Registrar");
            case "Admin": return RedirectToAction("Dashboard", "Admin");
```

```
            case "Super Admin": return RedirectToAction("Dashboard", "SuperAdmin");
            default: return RedirectToAction("Index", "Home");
        }
    }

    public IActionResult Logout()
    {
        _logger.LogInformation($"AUTH_TERMINATION: Active user profile initialized manual
log-out pipeline.");
        HttpContext.Session.Clear();
        Response.Cookies.Delete(".AspNetCore.Session");
        return RedirectToAction("Login");
    }
  }
}
)
(using AxertLMS.Data;
using AxertLMS.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System;
using System.Linq;
using System.Text.Json;
using System.Threading.Tasks;
using AxertLMS.Helpers;

namespace AxertLMS.Controllers
{
  // ONLY Admins and Super Admins can access ANY method in this file now.
  // If a Student tries to type /Admin/Dashboard, the server rejects it instantly.
  [RoleGuard("Admin", "Super Admin")]
  public class AdminController : Controller
  {
    private readonly ApplicationDbContext _context;

    public AdminController(ApplicationDbContext context)
    {
      _context = context;
    }

    // ==========================================
    // 1. ADMIN DASHBOARD
    // ==========================================
    [HttpGet]
```

```
        public async Task<IActionResult> Dashboard()
        {
            // 1. Existing System Metrics
            ViewBag.TotalUsers = await _context.Users.CountAsync();
            ViewBag.ActiveUsers = await _context.Users.CountAsync(u => u.status == "Active");
            ViewBag.SuspendedUsers = await _context.Users.CountAsync(u => u.status ==
"Inactive");

            ViewBag.AdminCount = await _context.Users.CountAsync(u => u.role == "Admin" ||
u.role == "Super Admin");
            ViewBag.RegistrarCount = await _context.Users.CountAsync(u => u.role == "Registrar");
            ViewBag.TeacherCount = await _context.Users.CountAsync(u => u.role == "Teacher");
            ViewBag.StudentCount = await _context.Users.CountAsync(u => u.role == "Student");

            // ========================================
            // PROFESSIONAL API INTEGRATION
            // ========================================
            using var client = new HttpClient();

            // API 1: Network Security / IP Geolocation API (ip-api.com)
            try
            {
                var ipJson = await client.GetStringAsync("http://ip-api.com/json/");
                using var ipDoc = JsonDocument.Parse(ipJson);

                ViewBag.AdminIP = ipDoc.RootElement.GetProperty("query").GetString();
                ViewBag.AdminISP = ipDoc.RootElement.GetProperty("isp").GetString();
                ViewBag.AdminCity = ipDoc.RootElement.GetProperty("city").GetString();
            }
            catch
            {
                ViewBag.AdminIP = "Unknown";
                ViewBag.AdminISP = "Connection Error";
                ViewBag.AdminCity = "Unknown";
            }

            // API 2: Philippine Public Holidays API (Nager.Date)
            try
            {
                // Fetch all PH holidays for the current year
                int year = DateTime.Now.Year;
                var holidayJson = await
client.GetStringAsync($"https://date.nager.at/api/v3/PublicHolidays/{year}/PH");
                using var holidayDoc = JsonDocument.Parse(holidayJson);
```

```
            // Find the NEXT upcoming holiday safely
            var nextHoliday = holidayDoc.RootElement.EnumerateArray()
                .FirstOrDefault(h =>
                    h.TryGetProperty("date", out var dateProp) &&
                    DateTime.TryParse(dateProp.GetString(), out var parsedDate) &&
                    parsedDate >= DateTime.Now
                );

            if (nextHoliday.ValueKind != JsonValueKind.Undefined)
            {
                ViewBag.NextHolidayName = nextHoliday.GetProperty("localName").GetString();

                if (DateTime.TryParse(nextHoliday.GetProperty("date").GetString(), out var
holidayDate))
                {
                    ViewBag.NextHolidayDate = holidayDate.ToString("MMMM dd, yyyy");
                }
                else
                {
                    ViewBag.NextHolidayDate = "--";
                }
            }
            else
            {
                ViewBag.NextHolidayName = "No upcoming holidays";
                ViewBag.NextHolidayDate = "--";
            }
        }
        catch
        {
            ViewBag.NextHolidayName = "API Offline";
            ViewBag.NextHolidayDate = "--";
        }

        return View();
    } // <-- Fixed! This closing brace and return statement are now restored.

    // ==========================================
    // 2. MANAGE USERS (Global List)
    // ==========================================
    [HttpGet]
    public async Task<IActionResult> ManageUsers()
    {
```

```
            // Fetch ALL users, ordered by ID
            var allUsers = await _context.Users
                                 .OrderBy(u => u.user_id)
                                 .ToListAsync();
            return View(allUsers);
        }

        // ========================================
        // 3. THE "KILL SWITCH" (Toggle Active/Inactive)
        // ========================================
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> ToggleUserStatus(int user_id)
        {
            var user = await _context.Users.FindAsync(user_id);
            if (user != null)
            {
                // IT SECURITY FIX: Admins cannot touch Super Admins
                if (user.role == "Super Admin")
                {
                    TempData["ErrorMessage"] = "ACCESS DENIED: You do not have clearance to
alter a Super Admin account.";
                    return RedirectToAction("ManageUsers");
                }

                user.status = (user.status == "Active") ? "Inactive" : "Active";
                _context.Users.Update(user);
                await _context.SaveChangesAsync();

                TempData["SuccessMessage"] = $"User {user.email} status changed to
{user.status}.";
            }
            return RedirectToAction("ManageUsers");
        }

    }
}
)
(using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AxertLMS.Data;
using System.Linq;
using System.Threading.Tasks;
```

```
namespace AxertLMS.Controllers
{
    // The [ApiController] tag tells ASP.NET that this outputs raw JSON data, not HTML Views
    [Route("api/[controller]")]
    [ApiController]
    public class ApiController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public ApiController(ApplicationDbContext context)
        {
            _context = context;
        }

        // -----------------------------------------------------
        // API ENDPOINT 1: Get System Stats
        // URL to test: /api/api/stats
        // -----------------------------------------------------
        [HttpGet("stats")]
        public async Task<IActionResult> GetSystemStats()
        {
            var stats = new
            {
                TotalUsers = await _context.Users.CountAsync(),
                TotalStudents = await _context.Students.CountAsync(),
                TotalSections = await _context.Sections.CountAsync(),
                Status = "Online",
                ServerLocation = "Davao City"
            };

            return Ok(stats); // Returns a standard HTTP 200 OK JSON response
        }

        // -----------------------------------------------------
        // API ENDPOINT 2: Get Active Academic Sections
        // URL to test: /api/api/sections
        // -----------------------------------------------------
        [HttpGet("sections")]
        public async Task<IActionResult> GetSections()
        {
            // We select only the data we want to expose (Data Transfer Object pattern)
            var sections = await _context.Sections
                .Select(s => new {
                    s.section_id,
```

```
                s.section_name,
                s.strand,
                s.year_lvl
            })
            .ToListAsync();

        if (sections == null || !sections.Any())
        {
            return NotFound(new { message = "No sections found in the database." });
        }

        return Ok(sections);
    }
  }
}
)
(using Microsoft.AspNetCore.Mvc;

namespace AxertLMS.Controllers
{
  public class HomeController : Controller
  {
    public IActionResult Index()
    {
      return View();
    }
  }
}
)
(using AxertLMS.Data;
using AxertLMS.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
// Make sure to add your actual Data context namespace here if it's different!

namespace AxertLMS.Controllers
{
  public class ProfileController : Controller
  {
    private readonly ApplicationDbContext _context; // Replace 'YourDbContext' with your
actual DbContext name (e.g., AppDbContext)

    public ProfileController(ApplicationDbContext context)
    {
```

```
        _context = context;
    }

    public IActionResult Index()
    {
        // NOTE: Replace "101" with your actual session/authentication logic to get the logged-in
user's ID.
        // Example: int loggedInUserId = HttpContext.Session.GetInt32("UserId") ?? 0;
        int loggedInUserId = 101;

        var user = _context.Users.FirstOrDefault(u => u.user_id == loggedInUserId);

        if (user == null) return NotFound();

        var viewModel = new ProfileViewModel
        {
            UserId = user.user_id,
            Role = user.role,
            FirstName = user.f_name,
            LastName = user.l_name,
            Email = user.email,
            Status = user.status
        };

        // Fetch Role-Specific Data
        if (user.role == "Student")
        {
            var studentInfo = _context.Students
                .Include(s => s.Section)
                .FirstOrDefault(s => s.user_id == user.user_id);

            if (studentInfo != null)
            {
                viewModel.StudentNumber = studentInfo.student_number;
                viewModel.SectionName = studentInfo.Section?.section_name;
                viewModel.Strand = studentInfo.Section?.strand;
                viewModel.YearLevel = studentInfo.Section?.year_lvl;
                viewModel.EnrolledClassesCount = _context.Enrollments.Count(e => e.student_id
== studentInfo.student_id);
            }
        }
        else if (user.role == "Teacher")
        {
            var teacherInfo = _context.Teachers
```

```
                .FirstOrDefault(t => t.user_id == user.user_id);

            if (teacherInfo != null)
            {
                viewModel.Specialization = teacherInfo.specialization;
                viewModel.HandledClassesCount = _context.Classrooms.Count(c => c.teacher_id
== teacherInfo.teacher_id);
            }
        }

        return View(viewModel);
    }
  }
}
)
(using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AxertLMS.Data;
using AxertLMS.Models;
using AxertLMS.Services; // Connects to our new Services folder
using Microsoft.Extensions.Logging;

namespace AxertLMS.Controllers
{
  public class QuizGeneratorController : Controller
  {
    private readonly ApplicationDbContext _context;
    private readonly InternalQuizGeneratorService _quizGenerator; // We use our local service
now
    private readonly ILogger<QuizGeneratorController> _logger;

    // The constructor injects our local service instead of IConfiguration
    public QuizGeneratorController(
        ApplicationDbContext context,
        InternalQuizGeneratorService quizGenerator,
        ILogger<QuizGeneratorController> logger)
    {
        _context = context;
        _quizGenerator = quizGenerator;
        _logger = logger;
    }

    [HttpPost]
    public async Task<IActionResult> GenerateQuiz(int moduleId, string difficulty = "Easy")
```

```
    {
        // Find the module in the database
        var module = await _context.Modules
            .Include(m => m.Classroom)
            .ThenInclude(c => c!.Subject)
            .FirstOrDefaultAsync(m => m.module_id == moduleId);

        if (module == null)
            return NotFound();

        _logger.LogInformation("Generating internal quiz for Module ID: {ModuleId}", moduleId);

        // CALL YOUR INTERNAL LOGIC SERVICE (Completely Free!)
        List<QuizQuestionDto> questions = _quizGenerator.Generate(module.title,
module.description ?? "");

        // Validation Check: Ensure we actually created questions
        if (questions == null || !questions.Any())
        {
            return BadRequest("No quiz questions generated by the internal engine.");
        }

        // Create the Master Quiz Record
        var quiz = new Quiz
        {
            classroom_id = module.classroom_id,
            module_id = module.module_id,
            title = $"{module.title} Quiz",
            difficulty = difficulty,
            created_at = DateTime.Now
        };

        _context.Quizzes.Add(quiz);
        await _context.SaveChangesAsync(); // Saves the quiz header so we get an ID

        // Save each generated question into the Database
        foreach (var q in questions)
        {
            _context.QuizQuestions.Add(
                new QuizQuestion
                {
                    quiz_id = quiz.quiz_id,
                    question_type = q.question_type,
                    question_text = q.question_text,
```

```
                option_a = q.option_a,
                option_b = q.option_b,
                option_c = q.option_c,
                option_d = q.option_d,
                correct_answer = q.correct_answer
            });
        }

        await _context.SaveChangesAsync(); // Commit questions to the database

        // Take the user straight to the Quiz Details page
        return RedirectToAction("QuizDetails", new { id = quiz.quiz_id });
    }
    [HttpGet]
    public async Task<IActionResult> QuizDetails(int id)
    {
        // Fetch the quiz along with its generated questions from the database
        var quiz = await _context.Quizzes
            .Include(q => q.Questions)
            .Include(q => q.Module)
            .FirstOrDefaultAsync(q => q.quiz_id == id);

        if (quiz == null)
        {
            return NotFound();
        }

        return View(quiz); // This sends the data straight to the user interface
    }
  }
}
)
(using System;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AxertLMS.Data;
using AxertLMS.Models;
using AxertLMS.Helpers; // Fixes your CS0103 error for PasswordHelper!

namespace AxertLMS.Controllers
{
  [RoleGuard("Registrar", "Super Admin")]
```

```
public class RegistrarController : Controller
{
    private readonly ApplicationDbContext _context;

    public RegistrarController(ApplicationDbContext context)
    {
        _context = context;
    }

    // GET: Loads the form and populates the Section dropdown
    // GET: Loads the form and populates the dropdowns
    [HttpGet]
    public IActionResult RegisterStudent()
    {
        // IT FIX: Find "Pending Students" - Users with the role 'Student' but no entry in the
Students table yet
        var pendingUsers = _context.Users
            .Where(u => u.role == "Student" && !_context.Students.Any(s => s.user_id ==
u.user_id))
            .ToList();

        ViewBag.PendingUsers = pendingUsers;
        ViewBag.Sections = _context.Sections.ToList();

        return View();
    }

    // POST: Processes the official enrollment
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> RegisterStudent(int user_id, int section_id)
    {
        using var transaction = await _context.Database.BeginTransactionAsync();

        try
        {
            // 1. Generate an official Student Number based on the current year and total students
            int currentYear = DateTime.Now.Year;
            int studentCount = await _context.Students.CountAsync() + 1;
            string newStudentNumber = $"{currentYear}-{studentCount:D4}";

            // 2. Create the Official Student Record
            var newStudent = new Student
            {
```

```
                user_id = user_id,
                section_id = section_id,
                student_number = newStudentNumber
            };

            _context.Students.Add(newStudent);
            await _context.SaveChangesAsync();

            // 3. Auto-Enrollment Transaction
            var classrooms = await _context.Classrooms
                                    .Where(c => c.section_id == section_id)
                                    .ToListAsync();

            foreach (var room in classrooms)
            {
                var enrollment = new Enrollment
                {
                    student_id = newStudent.student_id,
                    classroom_id = room.classroom_id
                };
                _context.Enrollments.Add(enrollment);
            }

            await _context.SaveChangesAsync();
            await transaction.CommitAsync();

            // 4. Redirect safely back to the Dashboard
            TempData["SuccessMessage"] = $"Successfully enrolled student! Assigned ID:
{newStudentNumber}";
            return RedirectToAction("Dashboard");
        }
        catch (Exception ex)
        {
            await transaction.RollbackAsync();

            // Reload dropdowns if it fails
            ViewBag.PendingUsers = _context.Users.Where(u => u.role == "Student" &&
!_context.Students.Any(s => s.user_id == u.user_id)).ToList();
            ViewBag.Sections = _context.Sections.ToList();
            ViewBag.ErrorMessage = "Enrollment failed: " + ex.Message;

            return View();
        }
    }
```

```
        // ========================================
        // REGISTRAR DASHBOARD
        // ========================================
        [HttpGet]
        public async Task<IActionResult> Dashboard()
        {
            ViewBag.TotalStudents = await _context.Students.CountAsync();
            ViewBag.TotalTeachers = await _context.Teachers.CountAsync();
            ViewBag.TotalSections = await _context.Sections.CountAsync();
            ViewBag.TotalSubjects = await _context.Subjects.CountAsync();

            ViewBag.PendingStudents = await _context.Users
                .Where(u => u.role == "Student" &&
                            !_context.Students.Any(s => s.user_id == u.user_id))
                .CountAsync();

            var recentStudents = await _context.Students
                .Include(s => s.User)
                .Include(s => s.Section)
                .OrderByDescending(s => s.student_id)
                .Take(10)
                .ToListAsync();

            return View(recentStudents);
        }
        // ========================================
        // 1. REGISTER TEACHER
        // ========================================
        [HttpGet]
        public IActionResult RegisterTeacher()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> RegisterTeacher(string f_name, string l_name, string
email, string password, string specialization)
        {
            using var transaction = await _context.Database.BeginTransactionAsync();
            try
            {
                var newUser = new User
                {
```

```
            f_name = f_name,
            l_name = l_name,
            email = email,
            password = PasswordHelper.Hash(password),
            role = "Teacher",
            status = "Active"
        };

        _context.Users.Add(newUser);
        await _context.SaveChangesAsync();

        var newTeacher = new Teacher
        {
            user_id = newUser.user_id,
            specialization = specialization
        };

        _context.Teachers.Add(newTeacher);
        await _context.SaveChangesAsync();

        await transaction.CommitAsync();

        ViewBag.SuccessMessage = $"Successfully registered Faculty: {f_name} {l_name}!";
        return View();
    }
    catch (Exception ex)
    {
        await transaction.RollbackAsync();
        ViewBag.ErrorMessage = "Registration failed: " + ex.Message;
        return View();
    }
}

// ==========================================
// 2. MANAGE SECTIONS
// ==========================================
[HttpGet]
public async Task<IActionResult> ManageSections()
{
    var sections = await _context.Sections.ToListAsync();
    return View(sections);
}

[HttpPost]
```

```
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> AddSection(string section_name, string strand, string
year_lvl)
        {
            var newSection = new Section
            {
                section_name = section_name,
                strand = strand,
                year_lvl = year_lvl
            };
            _context.Sections.Add(newSection);
            await _context.SaveChangesAsync();
            return RedirectToAction("ManageSections");
        }

        // ========================================
        // 3. MANAGE SUBJECTS
        // ========================================
        [HttpGet]
        public async Task<IActionResult> ManageSubjects()
        {
            var subjects = await _context.Subjects.ToListAsync();
            return View(subjects);
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> AddSubject(string subject_name, string description)
        {
            var newSubject = new Subject
            {
                subject_name = subject_name,
                description = description
            };
            _context.Subjects.Add(newSubject);
            await _context.SaveChangesAsync();
            return RedirectToAction("ManageSubjects");
        }
    }
}
)
(using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Http;
using Microsoft.EntityFrameworkCore;
```

```
using AxertLMS.Data;
using AxertLMS.Models;
using System.IO;
using System.Linq;
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace AxertLMS.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;

        public StudentController(ApplicationDbContext context)
        {
            _context = context;
        }

        // --- YOUR ORIGINAL HELPER METHOD ---
        private bool IsStudent()
        {
            return HttpContext.Session.GetString("UserRole") == "Student";
        }

        // --- UPGRADED HCI DASHBOARD ---
        public IActionResult Dashboard()
        {
            if (!IsStudent()) return RedirectToAction("Login", "Account");

            string email = HttpContext.Session.GetString("UserEmail");
            if (email == null) return RedirectToAction("Login", "Account");

            var student = _context.Students
                .Include(s => s.User)
                .Include(s => s.Section)
                .FirstOrDefault(s => s.User != null && s.User.email == email);

            if (student == null) return NotFound("Student profile not found.");

            ViewBag.StudentName = student.User!.f_name;

            // Fetch Enrolled Classrooms
            var enrollments = _context.Enrollments
```

```
            .Where(e => e.student_id == student.student_id)
            .Include(e => e.Classroom).ThenInclude(c => c!.Subject)
            .Include(e => e.Classroom).ThenInclude(c => c!.Teacher).ThenInclude(t => t!.User)
            .ToList();

        var classroomIds = enrollments.Select(e => e.classroom_id).ToList();

        // Fetch ALL Assignments for their classes
        var allAssignments = _context.Assignments
            .Include(a => a.Classroom).ThenInclude(c => c!.Subject)
            .Where(a => classroomIds.Contains(a.classroom_id))
            .OrderBy(a => a.due_date)
            .ToList();

        // Fetch Submissions & Grades
        var mySubmissions = _context.Submissions.Where(s => s.student_id ==
student.student_id).ToList();
        var submissionIds = mySubmissions.Select(s => s.submission_id).ToList();
        var myGrades = _context.Grades.Where(g =>
submissionIds.Contains(g.submission_id)).ToList();

        // ========================================
        // HCI DATA AGGREGATION: The Universal To-Do List
        // ========================================
        var toDoItems = new List<dynamic>();
        int overdueCount = 0;
        int dueThisWeekCount = 0;

        foreach (var assignment in allAssignments)
        {
            var submission = mySubmissions.FirstOrDefault(s => s.assignment_id ==
assignment.assignment_id);
            bool isSubmitted = submission != null;

            // Store Philippine Time in a variable so we don't have to keep typing it
            DateTime phTime = DateTime.UtcNow.AddHours(8);

            bool isOverdue = !isSubmitted && assignment.due_date < phTime;

            if (isOverdue) overdueCount++;
            if (!isSubmitted && assignment.due_date >= phTime && assignment.due_date <=
phTime.AddDays(7)) dueThisWeekCount++;
            toDoItems.Add(new
            {
```

```
                AssignmentId = assignment.assignment_id,
                Title = assignment.title,
                Subject = assignment.Classroom!.Subject!.subject_name,
                DueDate = assignment.due_date,
                IsSubmitted = isSubmitted,
                IsOverdue = isOverdue
            });
        }

        ViewBag.ToDoList = toDoItems.Where(t => !t.IsSubmitted).Take(5).ToList(); // Top 5
urgent
        ViewBag.OverdueCount = overdueCount;
        ViewBag.DueThisWeekCount = dueThisWeekCount;

        // ========================================
        // VISUAL ANALYTICS: Graph Data
        // ========================================
        var chartLabels = new List<string>();
        var chartData = new List<double>();

        foreach (var enrollment in enrollments)
        {
            chartLabels.Add(enrollment.Classroom!.Subject!.subject_name!);

            var classGrades = myGrades
                .Where(g => mySubmissions.Any(s => s.submission_id == g.submission_id &&
s.assignment_id == allAssignments.FirstOrDefault(a => a.classroom_id ==
enrollment.classroom_id)?.assignment_id))
                .Select(g => (double)g.score)
                .ToList();

            double average = classGrades.Any() ? classGrades.Average() : 0;
            chartData.Add(Math.Round(average, 2));
        }

        ViewBag.ChartLabels = chartLabels;
        ViewBag.ChartData = chartData;
        ViewBag.Enrollments = enrollments;

        // Note: Returning 'enrollments' so it matches the @model in the Dashboard.cshtml view
I gave you
        return View(enrollments);
    }
```

```
    // --- ALL YOUR ORIGINAL METHODS PRESERVED BELOW ---

    public IActionResult Classrooms()
    {
        if (!IsStudent()) return RedirectToAction("Login", "Account");
        var userEmail = HttpContext.Session.GetString("UserEmail");

        var student = _context.Students
            .Include(s => s.User)
            .Include(s => s.Section)
            .FirstOrDefault(s => s.User!.email == userEmail);

        var enrolledClasses = _context.Enrollments
            .Where(e => e.student_id == student!.student_id)
            .Include(e => e.Classroom).ThenInclude(c => c.Subject)
            .Include(e => e.Classroom).ThenInclude(c => c.Teacher).ThenInclude(t => t.User)
            .Include(e => e.Classroom).ThenInclude(c => c.Section)
            .Select(e => e.Classroom)
            .ToList();

        return View(enrolledClasses);
    }

    public IActionResult ClassDetails(int id)
    {
        if (!IsStudent()) return RedirectToAction("Login", "Account");
        var userEmail = HttpContext.Session.GetString("UserEmail");

        var student = _context.Students.Include(s => s.User).FirstOrDefault(s => s.User.email
== userEmail);

        bool isEnrolled = _context.Enrollments.Any(e => e.classroom_id == id && e.student_id
== student.student_id);
        if (!isEnrolled) return Unauthorized("You are not enrolled in this class.");

        var currentStudent = _context.Students.FirstOrDefault(s => s.User!.email == userEmail);

        var classroom = _context.Classrooms
            .Include(c => c.Subject)
            .Include(c => c.Teacher).ThenInclude(t => t.User)
            .Include(c => c.Section)
            .FirstOrDefault(c => c.classroom_id == id);

        ViewBag.Announcements = _context.Announcements
```

```
            .Include(a => a.User)
            .Where(a => a.classroom_id == id)
            .OrderByDescending(a => a.created_at)
            .ToList();

        ViewBag.Modules = _context.Modules.Where(m => m.classroom_id == id).ToList();

        ViewBag.Assignments = _context.Assignments
            .Where(a => a.classroom_id == id)
            .OrderBy(a => a.due_date)
            .ToList();

        ViewBag.Classmates = _context.Students
            .Include(s => s.User)
            .Where(s => s.section_id == classroom!.section_id)
            .OrderBy(s => s.User!.l_name)
            .ToList();

        if (currentStudent != null)
        {
            ViewBag.SubmittedAssignmentIds = _context.Submissions
                .Where(s => s.student_id == currentStudent.student_id &&
s.Assignment!.classroom_id == id)
                .Select(s => s.assignment_id)
                .ToList();
        }

        return View(classroom);
    }

    [HttpGet]
    public IActionResult SubmitAssignment(int id)
    {
        if (!IsStudent()) return RedirectToAction("Login", "Account");
        var userEmail = HttpContext.Session.GetString("UserEmail");

        var student = _context.Students.FirstOrDefault(s => s.User!.email == userEmail);
        if (student == null) return NotFound();

        var assignment = _context.Assignments
            .Include(a => a.Classroom)
            .ThenInclude(c => c.Subject)
            .FirstOrDefault(a => a.assignment_id == id);
```

```
        if (assignment == null) return NotFound();

        var existingSubmission = _context.Submissions
            .FirstOrDefault(s => s.assignment_id == id && s.student_id == student.student_id);

        ViewBag.HasSubmitted = existingSubmission != null;
        ViewBag.SubmissionDetails = existingSubmission;

        return View(assignment);
    }

    [HttpPost]
    public async Task<IActionResult> UploadSubmission(int assignment_id, int classroom_id,
IFormFile studentFile)
    {
        if (!IsStudent()) return RedirectToAction("Login", "Account");
        var userEmail = HttpContext.Session.GetString("UserEmail");

        var student = _context.Students.FirstOrDefault(s => s.User!.email == userEmail);
        if (student == null) return NotFound();

        if (studentFile == null || studentFile.Length == 0)
        {
            TempData["Error"] = "Please select a file to upload.";
            return RedirectToAction("SubmitAssignment", new { id = assignment_id });
        }

        var uploadsFolder = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot",
"uploads", "submissions");
        if (!Directory.Exists(uploadsFolder))
        {
            Directory.CreateDirectory(uploadsFolder);
        }

        var uniqueFileName =
$"{student.student_id}_{assignment_id}_{Guid.NewGuid().ToString().Substring(0,
6)}_{Path.GetFileName(studentFile.FileName)}";
        var filePath = Path.Combine(uploadsFolder, uniqueFileName);

        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            await studentFile.CopyToAsync(stream);
        }
```

```
            var newSubmission = new Submission
            {
                assignment_id = assignment_id,
                student_id = student.student_id,
                file_path = $"/uploads/submissions/{uniqueFileName}",
                submitted_at = DateTime.Now
            };

            _context.Submissions.Add(newSubmission);
            await _context.SaveChangesAsync();

            return RedirectToAction("SubmitAssignment", new { id = assignment_id });
        }

        public IActionResult Assignments()
        {
            if (!IsStudent()) return RedirectToAction("Login", "Account");
            var userEmail = HttpContext.Session.GetString("UserEmail");

            var student = _context.Students.Include(s => s.User).FirstOrDefault(s => s.User!.email
== userEmail);
            if (student == null) return NotFound();

            var enrolledClassIds = _context.Enrollments
                .Where(e => e.student_id == student.student_id)
                .Select(e => e.classroom_id)
                .ToList();

            var allAssignments = _context.Assignments
                .Include(a => a.Classroom)
                    .ThenInclude(c => c.Subject)
                .Where(a => enrolledClassIds.Contains(a.classroom_id))
                .OrderBy(a => a.due_date)
                .ToList();

            ViewBag.SubmittedIds = _context.Submissions
                .Where(s => s.student_id == student.student_id)
                .Select(s => s.assignment_id)
                .ToList();

            return View(allAssignments);
        }

        public IActionResult Announcements()
```

```
    {
        if (!IsStudent()) return RedirectToAction("Login", "Account");
        var userEmail = HttpContext.Session.GetString("UserEmail");

        var student = _context.Students.Include(s => s.User).FirstOrDefault(s => s.User!.email
== userEmail);
        if (student == null) return NotFound();

        var enrolledClassIds = _context.Enrollments
            .Where(e => e.student_id == student.student_id)
            .Select(e => e.classroom_id)
            .ToList();

        var globalAnnouncements = _context.Announcements
            .Include(a => a.User)
            .Include(a => a.Classroom)
                .ThenInclude(c => c.Subject)
            .Where(a => enrolledClassIds.Contains(a.classroom_id))
            .OrderByDescending(a => a.created_at)
            .ToList();

        return View(globalAnnouncements);
    }

    public IActionResult Calendar(string targetDate)
    {
        if (!IsStudent()) return RedirectToAction("Login", "Account");
        var userEmail = HttpContext.Session.GetString("UserEmail");

        var student = _context.Students.Include(s => s.User).FirstOrDefault(s => s.User!.email
== userEmail);
        if (student == null) return NotFound();

        DateTime currentDate = DateTime.Today;
        if (!string.IsNullOrEmpty(targetDate))
        {
            DateTime.TryParse(targetDate, out currentDate);
        }

        int diff = (7 + (currentDate.DayOfWeek - DayOfWeek.Monday)) % 7;
        DateTime startOfWeek = currentDate.AddDays(-1 * diff).Date;
        DateTime endOfWeek =
startOfWeek.AddDays(6).Date.AddHours(23).AddMinutes(59).AddSeconds(59);
```

```
            var enrolledClassIds = _context.Enrollments
                .Where(e => e.student_id == student.student_id)
                .Select(e => e.classroom_id)
                .ToList();

            var weeklyAssignments = _context.Assignments
                .Include(a => a.Classroom)
                    .ThenInclude(c => c.Subject)
                .Where(a => enrolledClassIds.Contains(a.classroom_id)
                        && a.due_date >= startOfWeek
                        && a.due_date <= endOfWeek)
                .OrderBy(a => a.due_date)
                .ToList();

            ViewBag.StartOfWeek = startOfWeek;
            ViewBag.EndOfWeek = endOfWeek;
            ViewBag.PrevWeek = startOfWeek.AddDays(-7).ToString("yyyy-MM-dd");
            ViewBag.NextWeek = startOfWeek.AddDays(7).ToString("yyyy-MM-dd");

            ViewBag.SubmittedIds = _context.Submissions
                .Where(s => s.student_id == student.student_id)
                .Select(s => s.assignment_id)
                .ToList();

            return View(weeklyAssignments);
        }
    }
}
)
(using AxertLMS.Data;
using AxertLMS.Helpers;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Linq;
using System.Threading.Tasks;
using AxertLMS.Helpers;

namespace AxertLMS.Controllers
{
    [RoleGuard("Super Admin")]
    public class SuperAdminController : Controller
    {
        private readonly ApplicationDbContext _context;
```

```
public SuperAdminController(ApplicationDbContext context)
{
    _context = context;
}

[HttpGet]
public async Task<IActionResult> Dashboard()
{
    // Super Admins care about raw data weight and backend health
    ViewBag.TotalDbRows = await _context.Users.CountAsync() +
                          await _context.Students.CountAsync() +
                          await _context.Enrollments.CountAsync() +
                          await _context.Submissions.CountAsync();

    ViewBag.ActiveAdmins = await _context.Users.CountAsync(u => u.role == "Admin");

    // Check if database connection is alive
    ViewBag.DbStatus = _context.Database.CanConnect() ? "ONLINE" : "OFFLINE";

    return View();
}
// ========================================
// SUPER ADMIN: ROLE MANAGER
// ========================================
[HttpGet]
public async Task<IActionResult> RoleManager(string searchQuery)
{
    var usersQuery = _context.Users.AsQueryable();

    // IT Fix: Added a search bar so they can actually find the user!
    if (!string.IsNullOrEmpty(searchQuery))
    {
        usersQuery = usersQuery.Where(u => u.l_name.Contains(searchQuery) ||
                                           u.f_name.Contains(searchQuery) ||
                                           u.email.Contains(searchQuery));
    }

    var users = await usersQuery.OrderBy(u => u.role).ToListAsync();
    return View(users);
}

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> ChangeRole(int user_id, string new_role)
```

```
        {
            var user = await _context.Users.FindAsync(user_id);
            if (user != null)
            {
                // Safety: Prevent Super Admin from accidentally demoting themselves and locking
themselves out
                var currentSessionUserId = HttpContext.Session.GetInt32("UserId");
                if (user.user_id == currentSessionUserId)
                {
                    TempData["ErrorMessage"] = "CRITICAL: You cannot alter your own Super Admin
access.";
                    return RedirectToAction("RoleManager");
                }

                user.role = new_role;
                _context.Users.Update(user);
                await _context.SaveChangesAsync();
                TempData["SuccessMessage"] = $"System Access for {user.email} updated to
[{new_role}].";
            }
            return RedirectToAction("RoleManager");
        }
        // ==========================================
        // SUPER ADMIN: RAW DATABASE MONITOR
        // ==========================================
        [HttpGet]
        public async Task<IActionResult> Database()
        {
            // Security Check: Ensure only Super Admins can access this raw data
            var currentRole = HttpContext.Session.GetString("UserRole");
            if (currentRole != "Super Admin")
            {
                return RedirectToAction("Login", "Account");
            }

            // Fetch metadata about the Database Connection
            ViewBag.DbProvider = _context.Database.ProviderName;
            ViewBag.CanConnect = _context.Database.CanConnect() ? "CONNECTED" :
"DISCONNECTED";

            // Map out the exact row count of every table in the schema
            ViewBag.TableStats = new Dictionary<string, int>
            {
                { "Users", await _context.Users.CountAsync() },
```

```
            { "Students", await _context.Students.CountAsync() },
            { "Teachers", await _context.Teachers.CountAsync() },
            { "Sections", await _context.Sections.CountAsync() },
            { "Subjects", await _context.Subjects.CountAsync() },
            { "Classrooms", await _context.Classrooms.CountAsync() },
            { "Enrollments", await _context.Enrollments.CountAsync() },
            { "Announcements", await _context.Announcements.CountAsync() },
            { "Modules", await _context.Modules.CountAsync() },
            { "Assignments", await _context.Assignments.CountAsync() },
            { "Submissions", await _context.Submissions.CountAsync() },
            { "Grades", await _context.Grades.CountAsync() }
        };

        return View();
    }
  }
}
)
(using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Http;
using Microsoft.EntityFrameworkCore;
using AxertLMS.Data;
using AxertLMS.Models; // Ensure this points to where your ViewModel lives
using System.Linq;
using System.Collections.Generic;
using System.IO;
using System;
using System.Threading.Tasks;

namespace AxertLMS.Controllers
{
  public class TeacherController : Controller
  {
    private readonly ApplicationDbContext _context;

    public TeacherController(ApplicationDbContext context)
    {
      _context = context;
    }

    public IActionResult Dashboard()
    {
      // 1. Authenticate Teacher
      int? userId = HttpContext.Session.GetInt32("UserId");
```

```
if (userId == null || HttpContext.Session.GetString("UserRole") != "Teacher")
{
    return RedirectToAction("Login", "Account");
}

var teacher = _context.Teachers
    .Include(t => t.User)
    .FirstOrDefault(t => t.user_id == userId);

if (teacher == null) return RedirectToAction("Login", "Account");

// 2. Get the Teacher's Classrooms
var classrooms = _context.Classrooms
    .Include(c => c.Subject)
    .Include(c => c.Section)
    .Where(c => c.teacher_id == teacher.teacher_id)
    .ToList();

ViewBag.TeacherName = teacher.User!.l_name;

// 3. REAL DATA: Calculate Total Unique Students
var totalStudents = _context.Enrollments
    .Include(e => e.Classroom)
    .Where(e => e.Classroom!.teacher_id == teacher.teacher_id)
    .Select(e => e.student_id)
    .Distinct()
    .Count();
ViewBag.TotalStudents = totalStudents;

// 4. REAL DATA: Calculate Submissions that need grading
var totalUngraded = _context.Submissions
    .Include(s => s.Assignment)
    .ThenInclude(a => a!.Classroom)
    .Where(s => s.Assignment!.Classroom!.teacher_id == teacher.teacher_id)
    .Where(s => !_context.Grades.Any(g => g.submission_id == s.submission_id))
    .Count();
ViewBag.TotalUngraded = totalUngraded;

// 5. REAL DATA: Calculate Average Grade per Section for the Chart
var chartLabels = new List<string>();
var chartData = new List<double>();

foreach (var classroom in classrooms)
{
```

```
            chartLabels.Add(classroom.Section!.section_name!);

            var averageGrade = _context.Grades
                .Include(g => g.Submission)
                .ThenInclude(s => s!.Assignment)
                .Where(g => g.Submission!.Assignment!.classroom_id == classroom.classroom_id)
                .Average(g => (double?)g.score) ?? 0;

            chartData.Add(Math.Round(averageGrade, 2));
        }

        ViewBag.ChartLabels = chartLabels;
        ViewBag.ChartData = chartData;

        return View(classrooms);
    }

    // GET: /Teacher/ClassroomDetails/{id}
    [HttpGet]
    public async Task<IActionResult> ClassroomDetails(int id)
    {
        int? userId = HttpContext.Session.GetInt32("UserId");
        if (userId == null || HttpContext.Session.GetString("UserRole") != "Teacher")
            return RedirectToAction("Login", "Account");

        var teacher = await _context.Teachers.FirstOrDefaultAsync(t => t.user_id == userId);
        if (teacher == null) return RedirectToAction("Login", "Account");

        var classroom = await _context.Classrooms
            .Include(c => c.Subject)
            .Include(c => c.Section)
            .Include(c => c.Assignments)
            .Include(c => c.Modules)
            .FirstOrDefaultAsync(c => c.classroom_id == id && c.teacher_id ==
teacher.teacher_id);

        if (classroom == null) return RedirectToAction("Dashboard");

        var enrolledStudents = await _context.Enrollments
            .Include(e => e.Student)
                .ThenInclude(s => s!.User)
            .Where(e => e.classroom_id == id)
            .Select(e => e.Student)
            .ToListAsync();
```

```
            var announcements = await _context.Announcements
                .Include(a => a.User)
                .Where(a => a.classroom_id == id)
                .OrderByDescending(a => a.created_at)
                .ToListAsync();

            ViewBag.EnrolledStudents = enrolledStudents;
            ViewBag.Announcements = announcements;

            // Pass the Classroom model directly to the dashboard view
            return View(classroom);
        }

        [HttpGet]
        public IActionResult CreateAssignment(int id)
        {
            if (HttpContext.Session.GetString("UserRole") != "Teacher")
                return RedirectToAction("Login", "Account");

            ViewBag.ClassroomId = id;
            return View();
        }

        [HttpPost]
        public IActionResult CreateAssignment(int classroom_id, string title, DateTime due_date)
        {
            int? userId = HttpContext.Session.GetInt32("UserId");
            var teacher = _context.Teachers.FirstOrDefault(t => t.user_id == userId);
            if (teacher == null) return RedirectToAction("Login", "Account");

            var classroom = _context.Classrooms.FirstOrDefault(c => c.classroom_id ==
classroom_id && c.teacher_id == teacher.teacher_id);
            if (classroom == null) return RedirectToAction("Dashboard");

            if (string.IsNullOrWhiteSpace(title))
            {
                ModelState.AddModelError("", "Title is required.");
                ViewBag.ClassroomId = classroom_id;
                return View();
            }

            var newAssignment = new Assignment
            {
```

```
            classroom_id = classroom_id,
            title = title,
            due_date = due_date
        };

        _context.Assignments.Add(newAssignment);
        _context.SaveChanges();

        return RedirectToAction("ClassroomDetails", new { id = classroom_id });
    }

    [HttpGet]
    public async Task<IActionResult> AssignmentDetails(int id)
    {
        int? userId = HttpContext.Session.GetInt32("UserId");
        if (userId == null || HttpContext.Session.GetString("UserRole") != "Teacher")
            return RedirectToAction("Login", "Account");

        var teacher = await _context.Teachers.FirstOrDefaultAsync(t => t.user_id == userId);
        if (teacher == null) return RedirectToAction("Login", "Account");

        // Fetch the assignment details
        var assignment = await _context.Assignments
            .Include(a => a.Classroom)
            .FirstOrDefaultAsync(a => a.assignment_id == id && a.Classroom.teacher_id ==
teacher.teacher_id);

        if (assignment == null) return RedirectToAction("Dashboard");

        // Build out the grading queue matching your dynamic View logic
        var enrolledStudentIds = await _context.Enrollments
            .Where(e => e.classroom_id == assignment.classroom_id)
            .Select(e => e.student_id)
            .ToListAsync();

        var students = await _context.Students
            .Include(s => s.User)
            .Where(s => enrolledStudentIds.Contains(s.student_id))
            .ToListAsync();

        var submissions = await _context.Submissions
            .Where(s => s.assignment_id == id)
            .ToListAsync();
```

```
            var grades = await _context.Grades

                .ToListAsync();

            // Constructing a dynamic matching block for the view layer layout
            var gradingQueue = new List<dynamic>();
            foreach (var student in students)
            {
                var submission = submissions.FirstOrDefault(s => s.student_id ==
student.student_id);
                Grade? grade = null;

                if (submission != null)
                {
                    grade = grades.FirstOrDefault(
                        g => g.submission_id == submission.submission_id);
                }

                gradingQueue.Add(new
                {
                    Student = student,
                    Submission = submission,
                    Grade = grade
                });
            }

            ViewBag.GradingQueue = gradingQueue;

            // Pass the Assignment model directly to the grading queue view
            return View(assignment);
        }

        [HttpPost]
        public IActionResult SaveGrade(int submission_id, int assignment_id, decimal score, string
feedback)
        {
            var existingGrade = _context.Grades.FirstOrDefault(g => g.submission_id ==
submission_id);

            if (existingGrade != null)
            {
                existingGrade.score = score;
                existingGrade.feedback = feedback;
            }
```

```
            else
            {
                _context.Grades.Add(new Grade
                {
                    submission_id = submission_id,
                    score = score,
                    feedback = feedback
                });
            }

            _context.SaveChanges();
            return RedirectToAction("AssignmentDetails", new { id = assignment_id });
        }

        [HttpPost]
        public IActionResult PostAnnouncement(int classroom_id, string message)
        {
            var userId = HttpContext.Session.GetInt32("UserId");
            var teacher = _context.Teachers.FirstOrDefault(t => t.user_id == userId);
            if (teacher == null) return RedirectToAction("Login", "Account");

            var classroom = _context.Classrooms.FirstOrDefault(c => c.classroom_id ==
classroom_id && c.teacher_id == teacher.teacher_id);
            if (classroom == null) return RedirectToAction("Dashboard");

            var newAnnouncement = new Announcement
            {
                classroom_id = classroom_id,
                user_id = userId.Value,
                message = message,
                created_at = DateTime.UtcNow.AddHours(8)
            };

            _context.Announcements.Add(newAnnouncement);
            _context.SaveChanges();

            return RedirectToAction("ClassroomDetails", new { id = classroom_id });
        }

        [HttpGet]
        public IActionResult UploadModule(int id)
        {
            if (HttpContext.Session.GetString("UserRole") != "Teacher")
                return RedirectToAction("Login", "Account");
```

```
            ViewBag.ClassroomId = id;
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> UploadModule(int classroom_id, string title, IFormFile
uploadedFile)
        {
            int? userId = HttpContext.Session.GetInt32("UserId");
            var teacher = _context.Teachers.FirstOrDefault(t => t.user_id == userId);
            if (teacher == null) return RedirectToAction("Login", "Account");

            var classroom = _context.Classrooms.FirstOrDefault(c => c.classroom_id ==
classroom_id && c.teacher_id == teacher.teacher_id);
            if (classroom == null) return RedirectToAction("Dashboard");

            if (uploadedFile != null && uploadedFile.Length > 0)
            {
                var uploadsFolder = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot",
"uploads", "modules");
                if (!Directory.Exists(uploadsFolder)) Directory.CreateDirectory(uploadsFolder);

                var uniqueFileName = Guid.NewGuid().ToString() + "_" +
Path.GetFileName(uploadedFile.FileName);
                var filePath = Path.Combine(uploadsFolder, uniqueFileName);

                using (var stream = new FileStream(filePath, FileMode.Create))
                {
                    await uploadedFile.CopyToAsync(stream);
                }

                var newModule = new Module
                {
                    classroom_id = classroom_id,
                    title = title,
                    file_path = "/uploads/modules/" + uniqueFileName,
                    created_at = DateTime.UtcNow.AddHours(8)
                };

                _context.Modules.Add(newModule);
                await _context.SaveChangesAsync();
            }
```

```
    return RedirectToAction("ClassroomDetails", new { id = classroom_id });
}

public IActionResult MyClasses()
{
    int? userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null || HttpContext.Session.GetString("UserRole") != "Teacher")
        return RedirectToAction("Login", "Account");

    var teacher = _context.Teachers.FirstOrDefault(t => t.user_id == userId);
    if (teacher == null) return RedirectToAction("Login", "Account");

    var classes = _context.Classrooms
        .Include(c => c.Subject)
        .Include(c => c.Section)
        .Where(c => c.teacher_id == teacher.teacher_id)
        .ToList();

    return View(classes);
}

public IActionResult GradingCenter()
{
    int? userId = HttpContext.Session.GetInt32("UserId");
    var teacher = _context.Teachers.FirstOrDefault(t => t.user_id == userId);
    if (teacher == null) return RedirectToAction("Login", "Account");

    var classrooms = _context.Classrooms
        .Include(c => c.Subject)
        .Include(c => c.Section)
        .Where(c => c.teacher_id == teacher.teacher_id)
        .ToList();

    var ungradedCounts = new Dictionary<int, int>();
    foreach (var cls in classrooms)
    {
        var count = _context.Submissions
            .Include(s => s.Assignment)
            .Where(s => s.Assignment!.classroom_id == cls.classroom_id)
            .Where(s => !_context.Grades.Any(g => g.submission_id == s.submission_id))
            .Count();
        ungradedCounts[cls.classroom_id] = count;
    }
```

```
            ViewBag.UngradedCounts = ungradedCounts;
            return View(classrooms);
        }

        // ==========================================
        // STEP 2: THE MASTER GRADEBOOK (Refactored)
        // ==========================================
        public IActionResult ClassGradebook(int id)
        {
            int? userId = HttpContext.Session.GetInt32("UserId");
            var teacher = _context.Teachers.FirstOrDefault(t => t.user_id == userId);
            if (teacher == null) return RedirectToAction("Login", "Account");

            var classroom = _context.Classrooms
                .Include(c => c.Subject)
                .Include(c => c.Section)
                .FirstOrDefault(c => c.classroom_id == id && c.teacher_id == teacher.teacher_id);

            if (classroom == null) return RedirectToAction("GradingCenter");

            var students = _context.Enrollments
                .Include(e => e.Student)
                .ThenInclude(s => s!.User)
                .Where(e => e.classroom_id == id)
                .Select(e => e.Student)
                .OrderBy(s => s!.User!.l_name)
                .ToList();

            var assignments = _context.Assignments
                .Where(a => a.classroom_id == id)
                .OrderBy(a => a.Category).ThenBy(a => a.due_date) // Assuming you have Category
mapped
                .ToList();

            var assignmentIds = assignments.Select(a => a.assignment_id).ToList();
            var submissions = _context.Submissions.Where(s =>
assignmentIds.Contains(s.assignment_id)).ToList();
            var submissionIds = submissions.Select(s => s.submission_id).ToList();
            var grades = _context.Grades.Where(g =>
submissionIds.Contains(g.submission_id)).ToList();

            // 🧠 CLEAN ARCHITECTURE FIX: Populate the ViewModel instead of ViewBag
            var viewModel = new ClassGradebookViewModel
            {
```

```
            Classroom = classroom,
            Students = students!,
            Assignments = assignments,
            Submissions = submissions,
            Grades = grades
        };

        return View(viewModel); // Strongly typed rendering
    }

    [HttpGet]
    public IActionResult EvaluateSubmission([FromQuery] int id)
    {
        int? userId = HttpContext.Session.GetInt32("UserId");
        if (userId == null || HttpContext.Session.GetString("UserRole") != "Teacher")
            return RedirectToAction("Login", "Account");

        var submission = _context.Submissions
            .Include(s => s.Student).ThenInclude(st => st!.User)
            .Include(s => s.Assignment)
            .FirstOrDefault(s => s.submission_id == id);

        if (submission == null) return RedirectToAction("GradingCenter");

        ViewBag.ExistingGrade = _context.Grades.FirstOrDefault(g => g.submission_id == id);

        return View(submission);
    }

    [HttpPost]
    public IActionResult EvaluateSubmission(int submission_id, decimal score, string
feedback)
    {
        int? userId = HttpContext.Session.GetInt32("UserId");
        if (userId == null) return RedirectToAction("Login", "Account");

        var submission = _context.Submissions.Include(s => s.Assignment).FirstOrDefault(s =>
s.submission_id == submission_id);
        if (submission == null) return RedirectToAction("GradingCenter");

        var existingGrade = _context.Grades.FirstOrDefault(g => g.submission_id ==
submission_id);

        if (existingGrade != null)
```

```
            {
                existingGrade.score = score;
                existingGrade.feedback = feedback;
            }
            else
            {
                _context.Grades.Add(new Grade
                {
                    submission_id = submission_id,
                    score = score,
                    feedback = feedback
                });
            }

            _context.SaveChanges();

            return RedirectToAction("ClassGradebook", new { id =
submission.Assignment!.classroom_id });
        }

        [HttpGet]
        public IActionResult AllAssignment()
        {
            int? userId = HttpContext.Session.GetInt32("UserId");

            if (userId == null || HttpContext.Session.GetString("UserRole") != "Teacher")
            {
                return RedirectToAction("Login", "Account");
            }

            var teacher = _context.Teachers.FirstOrDefault(t => t.user_id == userId);
            if (teacher == null)
            {
                return RedirectToAction("Login", "Account");
            }

            var assignments = _context.Assignments
                .Include(a => a.Classroom)
                    .ThenInclude(c => c.Subject)
                .Include(a => a.Classroom)
                    .ThenInclude(c => c.Section)
                .Where(a => a.Classroom.teacher_id == teacher.teacher_id)
                .OrderByDescending(a => a.due_date)
                .ToList();
```

```
            return View(assignments);
        }

        public IActionResult QuizDetails(int id)
        {
            int? userId = HttpContext.Session.GetInt32("UserId");
            if (userId == null || HttpContext.Session.GetString("UserRole") != "Teacher")
            {
                return RedirectToAction("Login", "Account");
            }

            var teacher = _context.Teachers.FirstOrDefault(t => t.user_id == userId);
            if (teacher == null) return RedirectToAction("Login", "Account");

            var quiz = _context.Quizzes
                .Include(q => q.Questions)
                .Include(q => q.Classroom)
                .FirstOrDefault(q => q.quiz_id == id);

            if (quiz == null)
                return RedirectToAction("Dashboard");

            if (quiz.Classroom!.teacher_id != teacher.teacher_id)
            {
                return RedirectToAction("Dashboard");
            }

            return View(quiz);
        }
    }
}
)

FullName
--------
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\AccountController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\AdminController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\AnalyticsController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\ApiController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\HomeController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\ProfileController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\QuizGeneratorController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\RegistrarController.cs
```

D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\StudentController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\SuperAdminController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Controllers\TeacherController.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Data\Migrations\00000000000000_CreateIdentitySchema.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Data\Migrations\00000000000000_CreateIdentitySchema.Designer.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Data\Migrations\20260327085733_AddDepEdGradingFields.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Data\Migrations\20260327085733_AddDepEdGradingFields.Designer.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Data\Migrations\20260604125239_AddQuizTablesFinal.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Data\Migrations\20260604125239_AddQuizTablesFinal.Designer.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Data\Migrations\ApplicationDbContextModelSnapshot.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Data\ApplicationDbContext.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Helpers\PasswordHelper.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Helpers\RoleGuardAttribute.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Announcement.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Assignment.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\ClassGradebookViewModel.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Classroom.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Enrollment.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\ErrorViewModel.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Grade.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Module.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\ProfileViewModel.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Quiz.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\QuizQuestion.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Section.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Student.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\StudentViewModel.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Subject.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Submission.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\Teacher.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Models\User.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\obj\Debug\net10.0\.NETCoreApp,Version=v10.0.AssemblyAttributes.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\obj\Debug\net10.0\AxertLMS.AssemblyInfo.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\obj\Debug\net10.0\AxertLMS.GlobalUsings.g.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\obj\Debug\net10.0\AxertLMS.MvcApplicationPartsAssemblyInfo.cs

D:\repos\Axert_LMS\AxertLMS\AxertLMS\obj\Debug\net10.0\AxertLMS.RazorAssemblyInfo.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Services\InternalPlagiarismService.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Services\InternalQuizGeneratorService.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Account\Login.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Account\Login.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Admin\Dashboard.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Admin\Dashboard.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Admin\ManageUsers.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Admin\ManageUsers.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Analytics\ClassAnalytics.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Analytics\ClassAnalytics.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Home\Index.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Home\Privacy.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Profile\Index.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Profile\Index.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\QuizGenerator\Generate.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\QuizGenerator\Generate.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\QuizGenerator\QuizDetails.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\QuizGenerator\QuizDetails.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\QuizGenerator\ViewQuiz.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\QuizGenerator\ViewQuiz.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\Dashboard.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\Dashboard.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\ManageSections.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\ManageSections.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\ManageSubjects.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\ManageSubjects.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\RegisterStudent.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\RegisterStudent.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\RegisterTeacher.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Registrar\RegisterTeacher.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Shared\Error.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Shared_Layout.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Shared_LoginLayout.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Shared_LoginLayout.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Shared_ValidationScriptsPartial.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Announcements.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Announcements.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Assignments.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Assignments.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Calendar.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Calendar.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\ClassDetails.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\ClassDetails.cshtml.cs

D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Classrooms.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Classrooms.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Dashboard.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\Dashboard.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\SubmitAssignment.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Student\SubmitAssignment.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\SuperAdmin\Dashboard.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\SuperAdmin\Dashboard.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\SuperAdmin\Database.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\SuperAdmin\Database.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\SuperAdmin\RoleManager.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\SuperAdmin\RoleManager.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\AllAssignment.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\AllAssignment.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\AssignmentDetails.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\AssignmentDetails.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\ClassGradebook.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\ClassGradebook.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\ClassroomDetails.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\ClassroomDetails.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\CreateAssignment.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\CreateAssignment.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\Dashboard.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\Dashboard.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\EvaluateSubmission.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\EvaluateSubmission.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\GradingCenter.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\GradingCenter.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\MyClasses.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\MyClasses.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\QuizDetails.cshtml
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Views\Teacher\QuizDetails.cshtml.cs
D:\repos\Axert_LMS\AxertLMS\AxertLMS\Program.cs